

1. Performance Testing

This report presents the results of performance testing conducted on the *Ticket Assignment Optimization System*, a platform designed to automatically route and assign customer support tickets to the most appropriate support agent based on predefined parameters such as skill level, workload, ticket priority, and SLA compliance.

The primary objective of the performance test was to ensure the system's **speed, scalability, stability, and efficiency** under various load conditions to maintain consistent support operations.

2. Objectives

The main objectives of performance testing were:

- To validate the system's response time for ticket creation, retrieval, and assignment.
- To assess system scalability under increasing ticket loads.
- To evaluate throughput and concurrency handling capability.
- To identify bottlenecks and potential points of degradation.
- To ensure SLA adherence under peak operational load.

3. Test Environment

Component	Details
Application Under Test (AUT)	Ticket Assignment Automation System
Version	v2.3.1
Server Configuration	8 vCPUs, 32 GB RAM, 500 GB SSD
Database	PostgreSQL 14
Load Generator Tool	Apache JMeter 5.6 / Locust 2.29
Operating System	Ubuntu 22.04 LTS
Network Bandwidth	1 Gbps internal test network
Concurrent Users Simulated	50, 100, 200, 500, 1000

4. Performance Metrics

Metric	Definition
Response Time (ms)	Time taken by the system to process and return a response.
Throughput (req/sec)	Number of requests processed per second.
CPU Utilization (%)	Percentage of CPU resources consumed during the test.
Memory Utilization (MB)	Amount of memory used under varying loads.
Error Rate (%)	Percentage of failed or timed-out requests.
Ticket Assignment Latency (s)	Time delay between ticket creation and assignment confirmation.

5. Test Scenarios

Scenario ID	Description	Expected Outcome
TS01	Ticket creation under concurrent user load	Response time < 2 sec
TS02	Ticket assignment automation (rule-based routing)	Assignment latency < 3 sec
TS03	Bulk ticket imports (1000+ tickets)	System remains stable
TS04	Agent availability recalculation under load	CPU < 80%
TS05	Ticket reassignment after escalation	Latency < 5 sec
TS06	System under sustained load for 2 hours	No crashes or memory leaks

6. Results Summary

Load (Users)	Avg Response Time (ms)	Throughput (req/sec)	CPU (%)	Memory (MB)	Error Rate (%)	Assignment Latency (s)
50	350	120	30	4200	0.0	1.2
100	420	210	45	4800	0.1	1.5
200	530	330	58	5400	0.2	1.9

Load (Users)	Avg Response Time (ms)	Throughput (req/sec)	CPU (%)	Memory (MB)	Error Rate (%)	Assignment Latency (s)
500	790	410	72	6100	0.5	2.6
1000	1180	500	88	7200	1.4	3.4

7. Observations & Analysis

- **Response Time:** The system maintained a sub-2-second response for up to 500 concurrent users. Beyond 800 users, response time degradation became noticeable.
 - **Throughput:** Increased linearly up to 500 users, after which database contention slightly affected throughput.
 - **CPU Utilization:** Peaked at 88% during the 1000-user test, suggesting the need for horizontal scaling or query optimization.
 - **Memory Utilization:** Grew steadily with load but remained within acceptable limits (below 8 GB usage).
 - **Error Rate:** Stayed below 1.5%, acceptable under stress conditions.
 - **Assignment Latency:** Met SLA targets (< 3s) up to 500 users; minor increase under extreme load.
-

8. Bottlenecks Identified

- **Database Write Contention:** Concurrent inserts during peak load increased latency.
 - **Thread Pool Saturation:** Under very high concurrency, thread pool exhaustion led to queue buildup.
 - **API Gateway Timeout:** Some requests hit the configured timeout (3s) threshold during peak load.
-

9. Recommendations

1. **Enable Database Connection Pooling** to reduce contention.
2. **Implement Caching Layer (Redis)** for repetitive ticket metadata requests.
3. **Optimize Query Execution Plans** and indexes in PostgreSQL.
4. **Increase Thread Pool Size** dynamically based on runtime load.

5. **Introduce Load Balancer Auto-Scaling** for backend microservices.
 6. **Conduct Endurance Testing** for 6+ hour continuous operation to validate stability.
-

10. Conclusion

The performance testing of the *Ticket Assignment Optimization System* demonstrates that the application is **highly efficient and scalable up to 500 concurrent users**, maintaining acceptable SLA targets for response and assignment times.

Minor performance degradation occurs beyond 800 users, primarily due to database contention and thread saturation. With recommended optimizations, the system can confidently support **enterprise-scale support operations** with minimal latency and high reliability.

11. Attachments

- **JMeter Test Plan (.jmx)**
- **Performance Graphs (Response Time, Throughput, CPU Usage)**
- **Raw Log Files and Error Reports**
- **Monitoring Dashboard Snapshots (Grafana / Prometheus)**



STREAMLINING TICKET ASSIGNMENT FOR EFFICIENT SUPPORT OPERATIONS

